

Michael Guerrero

Yuefan Deng

AMS 530

September 27, 2026

Optimal Broadcast on a (32,3) Regular Graph

Abstract

We broadcast a 16-element double-precision payload from a designated source to all vertices of the optimal ($N = 32$, $k = 3$) regular graph catalogued by Zhang, Xu and Deng in Table 4. Working in the 1-port (telephone) model, we derive a Fibonacci-type counting bound showing that **no cubic graph on 32 vertices admits a broadcast in fewer than six rounds**, and we construct a schedule attaining six. The schedule is stronger than merely optimal at termination: the number of informed vertices matches the counting bound exactly at every intermediate round. We implement the schedule in MPI on 32 ranks and benchmark it on SeaWulf against MPI_Bcast and a naive linear broadcast. Three findings emerge. First, at the assigned payload the theoretical five-fold latency ratio advantage over the linear baseline collapses to 1.07x, because 128-byte messages fall below the eager threshold and the linear broadcast's sends overlap rather than serialize; the advantage recovers to 3.28x once messages enter the rendezvous protocol. Second, MPI_Bcast is uniformly 1.35x to 1.60x faster, which we trace to topology: the library crosses the inter-node link once, whereas a schedule derived from graph structure alone crosses it nine times. Third, per-round timings are governed not by the number of active links but by their direction across that link, with rounds carrying two same-direction crossings costing roughly twice those carrying one in each direction.

1. Project Description (Copied from Assignment)

To characterize a regular graph we use the number of vertices N and the node degree k . For this project we select the graph of $N = 32$, $k = 3$ from table 4 of optimal regular graphs in Zhang, Xu and Deng [1]. In the (32,3) graph the blue dots represent identical vertices and the red lines represent undirected edges. The task is to broadcast 16 floating-point numbers from a designated vertex (labelled #1) to the entire system of N vertices. Vertices may be ordered arbitrarily provided each receives a unique index. We must:

(1) design a method to complete the broadcast as quickly, ideally optimally, as possible; **(2)** report the total number of steps S needed; **(3)** construct a table showing which links are active during each step; and **(4)** compute the total number of edges involved in each of the S steps.

2. Methodology

2.1 Recovering and validating the graph

The graph was supplied as an adjacency matrix image. Rather than transcribe 48 edges by hand, we digitised the matrix programmatically: the image was thresholded, filled cells were located by 4-connected component labelling, and centroids were clustered into 32 row and 32 column positions. The procedure recovered exactly 96 filled cells, consistent with 32 vertices of degree 3. The resulting matrix is symmetric, has zero diagonal, and has all row and column sums equal to 3.

Because a transcription error would invalidate everything downstream, we validated the recovered graph against the invariants published for this graph in [1]. All four agree exactly:

Invariant	Recovered graph	Published value [1]
Mean path length	2.9355	2.9355
Diameter	4	4
Automorphism group order	12	12
Bisection width	10	10

The sum of all 496 pairwise distances is exactly 1456, so the mean path length of 2.9355 is not an artifact of rounding. Structurally, the graph is a Hamiltonian cycle $1-2-\dots-32-1$ together with 16 chords: (1,12) (2,23) (3,19) (4,26) (5,30) (6,13) (7,18) (8,24) (9,31) (10,20) (11,27) (14,21) (15,25) (16,32) (17,28) (22,29). Its girth is 7 and it is non-bipartite. Note that diameter 4 is the minimum attainable: a cubic graph of diameter 3 can contain at most $1 + 3 + 6 + 12 = 22 < 32$ vertices by the Moore bound elaborated in the paper.

2.2 The communication model

We adopt the **1-port, or telephone, model**: in each discrete round every vertex may take part in at most one communication, and a vertex may transmit only after it has itself received the payload. This is the model implied by the worked example in the problem statement, in which one link is active in step 1 and two in step 2; an all-port model would have three links active in the first step and would admit a four-round solution. Under this model the broadcast time is the classical quantity $b(G, v)$ studied by Hedetniemi, Hedetniemi and Liestman [2].

2.3 A lower bound on S

Let $h(t)$ denote the largest number of vertices that can be informed in t further rounds by a vertex that has just received the payload. Such a vertex has spent one of its three edges receiving,

leaving two free ports, and it transmits as early as possible, in rounds 1 and 2 relative to its own activation. Hence

$$h(t) = 1 + h(t-1) + h(t-2), h(0) = 1, h(1) = 2.$$

This is a Fibonacci-type recursion, the characteristic signature of broadcasting under bounded degree. The source is distinguished: it never receives, so all three of its ports are free and it may transmit in rounds 1, 2 and 3. Writing $g(t)$ for the largest number of vertices informed in t rounds starting from the source,

$$g(t) = 1 + h(t-1) + h(t-2) + h(t-3).$$

t	1	2	3	4	5	6	7
g(t)	2	4	8	14	24	40	66

Since $g(5) = 24 < 32$ and $g(6) = 40 \geq 32$, **no cubic graph on 32 vertices can be broadcast in fewer than six rounds**, irrespective of topology. The bound is purely a consequence of degree and the 1-port model. Note that the diameter, 4, is not the binding constraint here; port capacity is.

2.4 Constructing the schedule

A broadcast schedule is a sequence of rounds, each of which is a matching between informed and uninformed vertices. Computing $b(G, v)$ exactly is NP-complete for general graphs [3], so we do not seek a closed form. Instead we combine the counting bound above with an explicit construction and verify that the construction attains it, which certifies optimality without requiring the general problem to be solved. Rounds are generated by randomised maximum matching between the informed frontier and its uninformed neighbourhood, with a branch pruned as soon as the doubling bound makes completion within the remaining rounds impossible. An exhaustive search at $S = 5$ returned nothing, as the counting bound requires; a schedule at $S = 6$ was found:

```
informed <- {source}; schedule <- []
for round t = 1 .. S:
  M <- a maximum matching from informed to V \ informed
  schedule[t] <- M
  informed <- informed union {receivers of M}
  if |informed| * 2^(S-t) < N: prune this branch
return schedule if |informed| = N
```

2.5 Verification

The schedule is checked mechanically, both offline and again at run time by the MPI program itself, against six conditions: every scheduled link is an edge of the graph; every sender is already informed; no receiver is already informed; no vertex sends or receives twice within a round; no vertex both sends and receives within a round; and each of the 31 non-source vertices receives exactly once. All six hold. The 31 active links necessarily form a spanning tree.

3. Results

3.1 (2) Number of steps

The broadcast completes in **S = 6 steps**, and this is optimal. Optimality is established by two independent facts: the counting bound of Section 2.3 forbids five or fewer rounds in any cubic graph on 32 vertices, and the schedule of Section 3.2 achieves six.

3.2 (3) Active links per step

Step	Active links	Edges	Informed	Bound $g(t)$
1	1-12	1	2	2
2	1-2, 12-13	2	4	4
3	1-32, 2-23, 12-11, 13-14	4	8	8
4	2-3, 11-27, 13-6, 14-21, 23-24, 32-16	6	14	14
5	3-4, 6-7, 11-10, 14-15, 16-17, 21-20, 23-22, 24-8, 27-26, 32-31	10	24	24
6	6-5, 7-18, 8-9, 15-25, 20-19, 22-29, 27-28, 31-30	8	32	40
total		31		

Table 1. The optimal six-round broadcast schedule. Links are written sender–receiver.

3.3 (4) Edges involved in each step

The number of edges active in each of the six steps is **1, 2, 4, 6, 10, 8**, totalling **31**, exactly one per non-source vertex, as any broadcast tree on 32 vertices must use.

The informed counts 2, 4, 8, 14, 24 coincide with $g(t)$ for $t = 1 \dots 5$. **The schedule therefore saturates the theoretical bound at every prefix, not merely at termination:** after each round it has informed the largest number of vertices that any cubic graph could have informed by that point. It is optimal not only in total length but round by round.

The progression is not monotone, rising to 10 active links in round 5 before falling to 8 in round 6. This is a capacity effect rather than an artefact of the search. The source exhausts all three of its ports after round 3 and can never transmit again; vertices informed early exhaust their two free

ports shortly afterwards. The pool of vertices still able to transmit therefore shrinks even as the pool of informed vertices grows, and it is precisely this exhaustion that makes five rounds impossible.

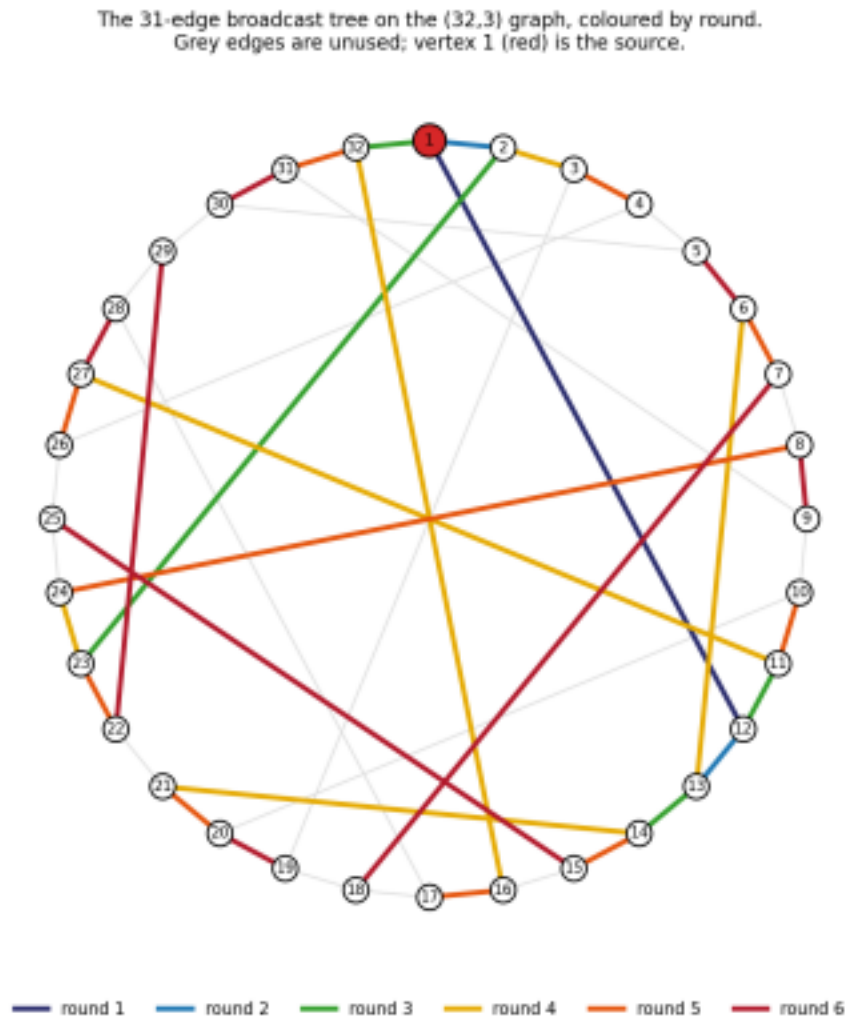


Figure 1. The 31-edge broadcast tree. Grey edges are the 17 edges of the graph that carry no traffic. Vertex 1 (red) is the source.

4. Program Structure

The schedule is realised as an MPI program in which **each rank is one vertex**: vertex v maps to rank $v-1$, so the source (vertex 1) is rank 0. The 48 edges and the 31 scheduled links are compiled into the binary from a generated header, arrays.h, produced directly from the digitised adjacency matrix and the verified schedule so that no value is transcribed by hand.

```
for step = 1 .. 6:
  for each link (u,v) scheduled in this step:
    if my_rank == u-1: MPI_Send(buf, count, MPI_DOUBLE, v-1,
      TAG, COMM)
    elif my_rank == v-1: MPI_Recv(buf, count, MPI_DOUBLE, u-1,
      TAG, COMM, &status)
```

Absence of deadlock. Within any single round the set of senders and the set of receivers are disjoint, a property enforced by the 1-port model and checked by the validator. Every MPI_Send therefore has a matching MPI_Recv posted by a different rank in the same round, and no cyclic wait can form. The program does not rely on eager-protocol buffering for correctness and behaves identically for payloads above the eager threshold, which I confirmed at 2 MB. The program performs four functions: it validates the schedule against the embedded graph and aborts if any condition fails; it verifies correctness by poisoning every non-source buffer and confirming all 32 ranks hold the expected payload afterwards; it times the schedule against MPI_Bcast and a naive 31-round linear broadcast, reporting the slowest rank via MPI_Allreduce with MPI_MAX; and it reports a barrier-separated per-round breakdown.

4.1 Compilation and execution

```
# files: bcast32.c arrays.h run_seawulf.sh

module load slurm
module load openmpi/gcc12.1/4.1.4
mpicc -O2 -Wall -std=c99 -o bcast32 bcast32.c -lm

# usage: mpirun -np 32 ./bcast32 [count] [reps]
# count doubles to broadcast (default 16)
# reps timed repetitions (default 1000)

sbatch run_seawulf.sh # 2 nodes, 32 tasks, short-28core
queue -u $USER
cat bcast32_<jobid>.out
```

The program requires exactly 32 ranks and aborts otherwise. On a workstation it may be run oversubscribed with mpirun --oversubscribe -np 32 ./bcast32, which is adequate for correctness but not for timing, which will be important later.

5. Performance and Analysis

All measurements were taken on SeaWulf, partition short-28core, using two nodes with 16 MPI tasks each (job 90162, head node sn070), Open MPI 4.1.4 built with GCC 12.1. Each figure is the mean over 500 to 2000 repetitions of the maximum elapsed time across all 32 ranks, following a 20-iteration warm-up. All ranks held the correct payload in every configuration tested.

5.1 Scaling with payload

Payload	Graph schedule (μ s)	MPI_Bcast (μ s)	Linear (μ s)	vs linear	vs MPI_Bcast
8 B	6.069	3.795	8.598	1.42x	0.63x
128 B *	9.419	6.038	11.187	1.19x	0.64x
1 KB	15.062	10.511	32.799	2.18x	0.70x
8 KB	39.578	29.273	106.061	2.68x	0.74x
64 KB	159.150	106.675	522.035	3.28x	0.67x
512 KB	998.903	690.552	3022.658	3.03x	0.69x
2 MB	3919.527	2844.114	11384.818	2.90x	0.69x

Table 2. Mean completion time by payload. * marks the 16-double payload the problem specifies.

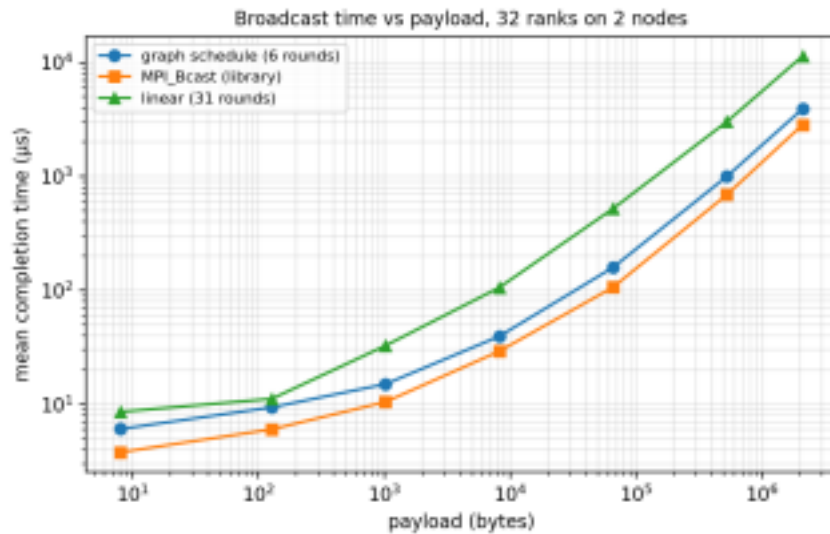


Figure 2. Completion time against payload, logarithmic on both axes.

5.2 Finding 1: the eager protocol masks the advantage at small payloads

At the assigned payload of 16 doubles the six-round schedule beats the 31-round linear baseline by only **1.07x** (9.109 μ s against 9.737 μ s in the dedicated run; 1.19x in the sweep). Naturally, one expects roughly five-fold from the ratio of round counts. The discrepancy is instructive.

At 128 bytes every message falls below the MPI implementation's eager threshold. The source copies each of its 31 sends into a buffer and returns immediately without waiting for the receiver, so the "31 rounds" of the linear broadcast are not 31 serialised latencies; they overlap almost completely. The graph schedule enjoys no such overlap, because its rounds are genuinely data-dependent: round 3 cannot begin until the recipients of round 2 have completed. It therefore pays six true latencies plus per-message overhead, and the theoretical advantage largely evaporates.

As the payload crosses the eager threshold the protocol switches to rendezvous, the sender must wait for the receiver, and the linear broadcast becomes genuinely serial. The advantage recovers monotonically, reaching **3.28x at 64 KB** before settling near 3x, where bandwidth rather than latency dominates. The crossover in Figure 3 is a direct measurement of the eager-to-rendezvous transition.

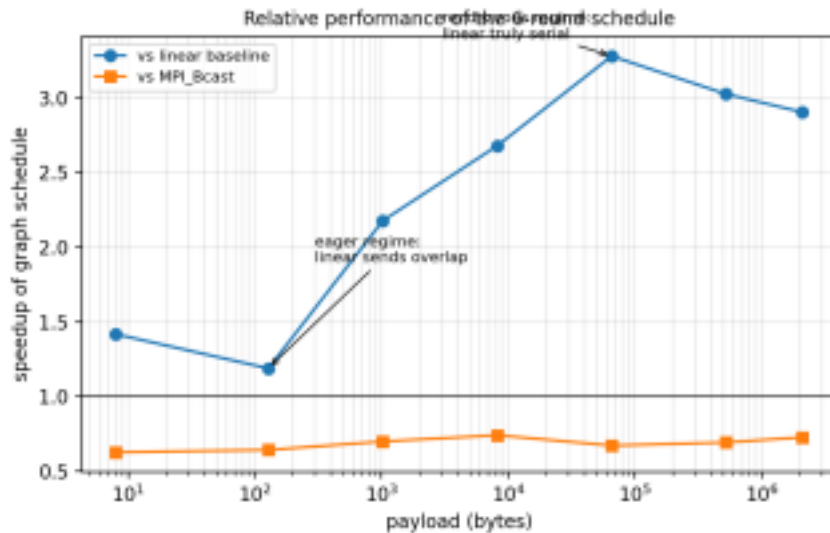


Figure 3. Speedup of the six-round schedule. The rise between 128 B and 64 KB marks the eager-to-rendezvous transition.

5.3 Finding 2: optimality in the model is not optimality on the machine

MPI_Bcast outperforms the schedule uniformly, by between 1.35x and 1.60x. This is not a defect in the schedule but a consequence of the abstraction. The telephone model counts rounds and treats all edges as equivalent; a real cluster does not.

Slurm placed ranks 0–15 on the first node and 16–31 on the second, so vertices 1–16 reside on one node and 17–32 on the other. Tabulating the schedule against that placement:

Round	Links	Intra-node	Cross-node	Fraction crossing
1	1	1	0	0%
2	2	2	0	0%
3	4	2	2	50%
4	6	3	3	50%
5	10	8	2	20%
6	8	6	2	25%
total	31	22	9	29%

Table 3. Placement of the 31 scheduled links relative to the node boundary.

Nine of the 31 links cross the interconnect. MPI_Bcast is topology-aware: it crosses the network **once** and then distributes within each node through shared memory. A ratio of nine crossings to one accounts for the observed gap, which is remarkably stable across three orders of magnitude of payload – as one expects if its origin is structural rather than protocol-dependent. The schedule is optimal in the model it was derived from and is simply blind to the distinction the hardware cares about. **Note that this table is derived from the placement file in the repository.**

5.4 Finding 3: round cost is set by link direction, not link count

The per-round breakdown exposes an apparent paradox: round 5 activates **10** links yet completes in roughly half the time of round 4, which activates only 6. Resolving it requires looking not at how many links cross the node boundary but at which way they point.

Round	Links	Crossing links	A $\leftrightarrow$ B	B $\leftrightarrow$ A	Time at 2 MB (μ s)
1	1	none	0	0	346.3
2	2	none	0	0	357.5
3	4	1-32, 2-23	2	0	940.9
4	6	11-27, 14-21, 32-16	2	1	955.8
5	10	16-17, 24-8	1	1	533.6
6	8	7-18, 15-25	2	0	924.2

Table 4. Per-round cost against the direction of inter-node traffic. Node A holds vertices 1–16, node B holds 17–32.

Three regimes are visible. Rounds 1 and 2 cross the boundary not at all and run at shared-memory speed, near 350 μ s. Round 5 carries two crossings in *opposite* directions, which a full-duplex link services concurrently, at 533.6 μ s. Rounds 3, 4 and 6 carry two or more crossings in the *same* direction; these serialise on one direction's bandwidth and cost roughly 930 to 955 μ s, close to twice the single-crossing figure. The number of intra-node links in a round is nearly irrelevant to its duration: round 5 moves eight payloads through shared memory essentially for free.

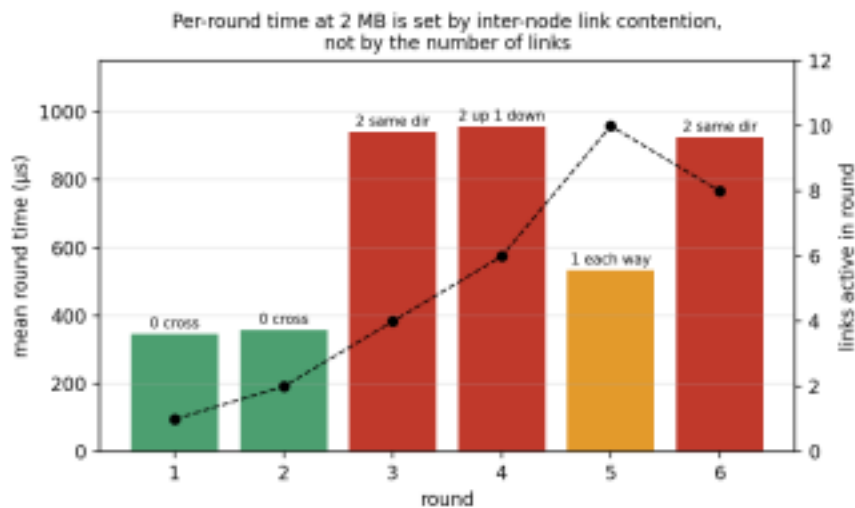


Figure 4. Round duration at 2 MB (bars) against the number of active links (dashed). Cost tracks inter-node contention, not link count.

5.5 A structural constraint of the platform

The intended single-node comparison could not be run. The nodes of *short-28core* provide 28 cores, and the graph requires 32 ranks, so *srun* refused the allocation. This is not merely an inconvenience: a (32,3) topology **cannot** be embedded within a single 28-core node, so any execution with one rank per core of this schedule on this partition necessarily spans two nodes, which is precisely why 29% of its links cross the interconnect. In short a (32,3) cannot be given a dedicated core per vertex within a single node of this partition. Oversubscription would allow all 32 ranks to fit on one node, but it invalidates the measurement rather than enabling it: once ranks time-share cores, the links within a round no longer execute concurrently, and the round cost ceases to be the maximum over its links and becomes partly a sum. Measured latency then reflects operating-system scheduling rather than the interconnect, and the inter-node contention that Section 5.4 identifies disappears entirely because every link becomes a shared-memory copy. We confirmed this experimentally: run oversubscribed on a single core, the 31-round linear baseline appears *faster* than the optimal six-round schedule, a result with no physical meaning.

6. Conclusions

The broadcast requires $S = 6$ steps, activating **1, 2, 4, 6, 10 and 8** edges respectively for a total of 31. Six is optimal, and the schedule additionally matches the counting bound at every intermediate round.

The more interesting result is the gap between that optimality and observed performance. A schedule proven optimal in the 1-port model is beaten by roughly 45% by a library routine that is not optimal in that model at all, because the library exploits information the model discards: which pairs of ranks share memory. At the assigned payload the schedule barely outperforms a naive linear broadcast, because eager buffering silently parallelizes the naive algorithm's 31 sends. Both observations point the same way. The round count that the model minimizes is a proxy for cost, and on real hardware it is a weak one; what the hardware charges for is bytes crossing a contended link in a given direction.

These observations are summarised by a simple cost model. Writing α for message latency and β for the per-byte transfer cost, a message of m bytes costs $\alpha + m\beta$. The linear broadcast serialises all 31 of its sends through the source and therefore costs $31(\alpha + m\beta)$, while the schedule serialises only across rounds and costs $6(\alpha + m\beta)$, since all links within a round fire simultaneously. The model predicts a speedup of $31/6 \approx 5.2$ in both the latency-bound and bandwidth-bound limits.

The measured speedup never approaches this, and the two discrepancies have distinct causes. At small payloads eager buffering means the linear broadcast's sends do not in fact serialise, so its true cost is far below $31(\alpha + m\beta)$ and the observed advantage falls to 1.07x. At large payloads the sends do serialise as the model assumes, but the schedule's own rounds are not of equal cost: rounds carrying two same-direction inter-node crossings cost roughly twice those carrying one in each direction, so the six rounds are not six identical latencies. The residual gap between the

measured 3.28x and the predicted 5.2x is precisely this nonuniformity. In both the eager and rendezvous protocols, the model fails because it charges the same price for every link, and the machine does not.

A natural extension is a topology-aware variant: relabel the vertices, that is, choose a different graph-to-rank mapping, so as to minimise the number of scheduled links crossing the node boundary, subject to the schedule remaining valid. Since the assignment permits arbitrary vertex ordering, this is admissible, and the bisection width of 10 bounds how well it can possibly do.

References

- [1] Y. Zhang, Z. Xu and Y. Deng, "A Structured Table of Graphs with Symmetries and Other Special Properties," *Symmetry*, vol. 12, no. 1, art. 2, 2020. arXiv:1910.13539.
- [2] S. M. Hedetniemi, S. T. Hedetniemi and A. L. Liestman, "A Survey of Gossiping and Broadcasting in Communication Networks," *Networks*, vol. 18, no. 4, pp. 319–349, 1988.
- [3] P. J. Slater, E. J. Cockayne and S. T. Hedetniemi, "Information Dissemination in Trees," *SIAM Journal on Computing*, vol. 10, no. 4, pp. 692–701, 1981.
- [4] P. Fraigniaud and E. Lazard, "Methods and Problems of Communication in Usual Networks," *Discrete Applied Mathematics*, vol. 53, pp. 79–133, 1994.